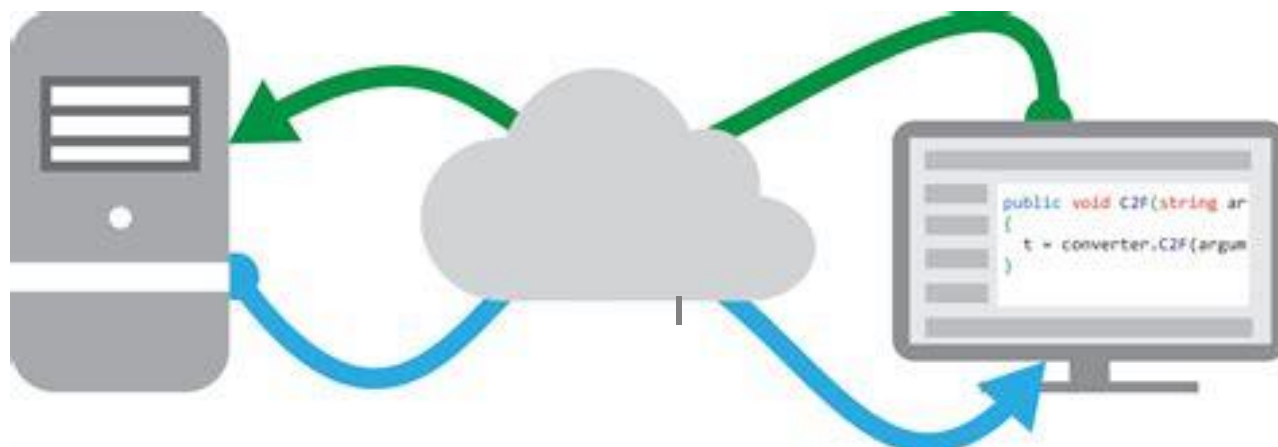


MANUAL DE WEBSERVICE



Inetws infor.net

Este documento describe los procedimientos de uso técnico del WebService

Contenido del Manual

¿A quién está dirigido?	3
¿Quiénes somos?	3
¿Cómo está estructurado el documento?.....	3
Introducción a Web Services.....	4
Web Service.....	4
¿Qué es XML, SOAP, WSDL, UDDI?	4
XML - eXtensible Markup Language.....	4
SOAP - Simple Object Access Protocol	6
WSDL - Web Services Description Language.....	8
Veamos un ejemplo de un documento WSDL:	8
Un poco de historia	10
¿Los Web Services pueden ser solo utilizados con HTTP?	11
Comencemos a Programar	15
Programación	15
Métodos de INETWS	19
Características de los métodos:	19
Cosas para tomar en consideración.	24
Conclusión	27
Apéndice:	28
Apéndice: Código soapUserNameToken.class	29
CODIGOS DE VERIFICACION ERROR	31

¿A quién está dirigido?

El presente documento está dirigido a las personas que ya poseen conocimientos sólidos sobre Internet, programación y quieren consumir el Web Service (servicio Web) INETWS de Infor.net

¿Quiénes somos?

La misión de infor.net es proveer al comercio, de estudios completos y actualizados, de cómodo acceso y fácil interpretación, a bajo costo y alto valor que aumente oportunidades y reduzca el riesgo en las operaciones.

Infor.net, empresa constituida en Islas Vírgenes Británicas, hace accesible la mayor información posible para permitir realizar análisis de créditos y otras negociaciones lícitas. Infor.net es un intermediario en el traslado de la información por lo que no califica a las personas.

Con más de 20 años de experiencia y la capacidad de elaborar los estudios más completos para personas individuales y empresas, somos el servicio más utilizado en Guatemala y con una importante y creciente presencia en el mercado centroamericano.

Los servicios de infor.net se ofrecen para Centroamérica por medio de colaboradores locales. Si desea más información puede comunicarse a los teléfonos que aparecen en la sección "Contáctenos".

¿Cómo está estructurado el documento?

Comenzaremos dando una introducción a lo que son los Web Services (Servicios Web), luego veremos un ejemplo de cómo utilizar el Web Service INETWS, para el cual utilizaremos PHP, y al finalizar definiremos los elementos que lo componen. En el apéndice pueden encontrar el código del ejemplo cliente.

Introducción a Web Services

Antes de comenzar con el uso del producto creo que es conveniente realizar una introducción a lo que son los Servicios Web, y para esto debemos comenzar con la definición de Web Service.

Web Service

Es un sistema software diseñado para soportar la interoperabilidad máquina - máquina a través de una red. Este tiene una interfaz descrita en un formato que puede ser procesado por una máquina (específicamente WSDL, que veremos más adelante). Otros sistemas interactúan con el Web Service utilizando mensajes SOAP los cuales se encuentran establecidos previamente. Entonces podríamos decir que un Web Service es una comunicación por medio de mensajes SOAP (son mensajes especiales que más adelante veremos detenidamente) entre diferentes equipos a través de una red.

¿Qué es XML, SOAP, WSDL, UDDI?

Para conocer cómo se realiza el intercambio de mensajes en los Web Services debemos primero saber cuáles son los elementos fundamentales que lo componen, estos son el XML, SOAP, WSDL, y UDDI.

XML - eXtensible Markup Language

Es un subconjunto simplificado del SGML el cual fue diseñado principalmente para documentos Web. Deja a los diseñadores crear sus propias “etiquetas” o “tags”

(Ej. <libro>), habilitando la definición, transmisión, validación y la interpretación de datos entre aplicaciones y en organizaciones.

Un punto que se considera importante aclarar es que el HTML y el XML tienen funciones diferentes:

- El HTML tiene por objeto mostrar información.
- XML = se ocupa de la información propiamente dicha (el contenido).



Este concepto es importante tenerlo en cuenta, ya que muchas personas al escuchar sobre XML piensan que es el sucesor de HTML.

Ejemplo de un documento XML sobre información de autos:

```
<?xml versión="1.1" encoding="UTF-8"?>
<vehículos>
  <coche>
    <marca>Toyota</marca>
    <modelo>Corolla</modelo>
    <fechaCompra>2002</fechaCompra>
  </coche>
  <coche>
    <marca>Honda</marca>
    <modelo>Civic</modelo>
    <fechaCompra>2003</fechaCompra>
  </coche>
```

Seguramente este tipo de archivos no sea una novedad para la mayoría de los que lean este artículo, ya que muchos documentos en Internet están en formato XML. Igualmente explicaremos brevemente las secciones del mismo:

<?xml versión="1.1" encoding="UTF-8"?>

Todo documento XML debe comenzar indicando que es un documento XML, la versión del mismo y su codificación. ¿Es por eso que se utiliza el tag <?xml?>

<vehículos>

Luego se indica un tag raíz (<vehículos>) el cual contendrá a los demás elementos. Es como en HTML el tag <HTML>. </HTML>, que dentro de él se encuentran los demás tags. Se lo conoce generalmente como root del documento.

<coche>

Luego definimos un tag <coche> el cual contendrá un coche en particular (en este caso solo incorpore 3 características de un coche: marca, modelo, y fecha de compra).

```
<marca>Toyota</marca>
<modelo>Corolla</modelo>
<fechaCompra>2002</fechaCompra>
```

Se definen 3 tags (marca, modelo, y fechaCompra), los cuales contienen los datos para un coche en particular.

```
<vehículos>
. <coche>. <marca>.
</marca>.
<modelo>. </modelo>
  <fechaCompra>. </fechaCompra>.
</coche>
</vehículos>
```

En el documento podemos apreciar que todos los tags que son abiertos <marca> deben ser cerrados </marca>. Esto es una exigencia del XML.

Se pueden anidar tags <coche><marca></marca>. </coche>. Marca está dentro del tag coche. El XML provee muchas otras posibilidades como puede ser que las etiquetas tengan atributos (<coche color="rojo">. </coche>), pero no nos detendremos en ellas ya que la idea es simplemente ofrecer los conceptos básicos para conocer de que se trata el mundo de los Web Services.

SOAP - Simple Object Access Protocol

Es un protocolo de comunicación, el cual permite la comunicación entre aplicaciones a través de mensajes por medio de Internet. Es independiente de la plataforma, y del lenguaje. Está basado en XML y es la base principal de los Web Services. Los mensajes SOAP son documento XML propiamente dicho, pero esto lo veremos más adelante cuando veamos un ejemplo de un mensaje SOAP.

Veamos cómo es la estructura básica del protocolo y la correspondiente explicación:

```
<?xml version="1.1"?><soap: Envelope xmlns: soap=http://www.w3.org/2001/12/soap-envelope Soap: encodingStyle="http://www.w3.org/2001/12/soap-encoding">
  </soap: Header> <soap:Body>
<soap: Fault>
  </soap:Fault>
</soap:Body>
</soap:Envelope>
```

<?xml version="1.1"?>

Como podemos ver en esta línea SOAP es un documento XML, y como tal, debe comenzar con el tag **<?xml?>** y la versión correspondiente.

<soap: Envelope

Aquí se indica que comienza el envelope (sobre) del mensaje.

xm:ns: soap = "http://www.w3.org/2001/12/soap-envelope"

Un mensaje SOAP debe contener siempre un elemento envelope asociado con el namespace (espacio de nombres) <http://www.w3.org/2001/12/soap-envelope>

Soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">

En esta línea lo que se hace es indicar donde se encuentran definidos los tipos de datos utilizados en el documento.

<soap: Header>

Esta línea indica el comienzo del Header (encabezado). En esta sección se incluye información específica del mensaje, como puede ser la autenticación.

</soap: Header>

Como todo documento XML los tags que son abiertos deben ser cerrados, esta línea indica la finalización del Header (encabezado).

<soap: Body>

Aquí comienza el cuerpo del mensaje, en esta sección se incorpora toda la información necesaria para el nodo final. Por ejemplo, los parámetros para la ejecución, o la respuesta a una petición.

<soap: Fault>

Cualquier tipo de fallo que se produzca será notificado en esta sección. La cual está contenida dentro del cuerpo del mensaje.

</soap:Fault>

Cierre de la sección Fault.

</soap:Body>

Indica el final del cuerpo del mensaje.

</soap:Envelope>

Fin del mensaje SOAP.

WSDL - Web Services Description Language

Es un protocolo basado en XML que describe los accesos al Web Service. Podríamos decir que es el manual de operación del Web Service, porque nos indica cuales son las interfaces que provee el Servicio Web y los tipos de datos necesarios para la utilización del mismo.

Veamos un ejemplo de un documento WSDL:

```
<?xml version="1.1">
<definitions>
  <types>
  </types>
  <message>

  </message>
  <portType>

  </portType>
  <binding>
  </binding>
</definitions>
```


<?xml version="1.1">

Este es otro documento XML, es por esto que debe comenzar con el tag <?xml .. ?>

<definitions>

Comienzo del documento, este tag agrupa a todos los demás.

<types>

Se definen los tipos de datos utilizados en el Web Service.

</types>

Fin de la definición de tipos.

<message>

Se definen los métodos y parámetros para realizar la operación. Cada mensaje puede consistir en una o más partes (parámetros).

</message>

Fin de la definición de los parámetros.

<portType>

Esta sección es la más importante, ya que se definen las operaciones que pueden ser realizadas, y los mensajes que involucran.

</portType>

Fin de la definición de las operaciones y mensajes.

<binding>

Se definen el formato del mensaje y detalles del protocolo para cada portType.

</binding>

Fin de la definición del formato del mensaje y detalles del protocolo para cada PortType.

</definitions>

Fin del documento WSDL

UDDI - Universal Discovery Description and Integration Es un modelo de directorios para Web Services. Es una especificación para mantener directorios estandarizados de información acerca de los Web Services, sus capacidades, ubicación y requerimientos en un formato reconocido universalmente. UDDI utiliza WSDL para describir las interfaces de los Web Services.

Es un lugar en el cual podemos buscar cuales son los Servicios Web disponibles, una especie de directorio en el cual podemos encontrar los Web Services publicados y publicar los Web Services que desarrollemos.

<http://soapclient.com/uddisearch.html>

<http://uddi.org/find.html>

Un poco de historia

Los Servicios Web surgieron ante una necesidad de estandarizar la comunicación entre distintas plataformas (PC, Mainframe, Mac, etc.) y lenguajes de programación (PHP, C#, Java, etc.).

Anteriormente se habían realizado intentos de crear estándares, pero fracasaron o no tuvieron el suficiente éxito, algunos de ellos son DCOM y CORBA, por ser dependientes de la implementación del vendedor DCOM - Microsoft, y CORBA - ORB (a pesar que CORBA de múltiples vendedores pueden operar entre sí, hay ciertas limitaciones para aplicaciones de niveles más altos en los cuales se necesite seguridad o administración de transacciones). Otro gran problema es que se hacía uso de RPC (Remote Procedure Call) para realizar la comunicación entre diferentes nodos, esto además de presentar ciertos problemas de seguridad, tiene la desventaja de que su implementación en un ambiente como es Internet, es casi imposible (muchos firewalls bloquean este tipo de mensajes, lo que hace prácticamente imposible a dos computadoras conectadas por Internet comunicarse). Los Web Services surgieron para finalmente poder lograr la tan esperada comunicación entre diferentes plataformas. En la actualidad muchos sistemas legacy están pasando a ser Web Services Es por esto que en 1999 se comenzó a plantear un nuevo estándar, el cual terminaría utilizando XML, SOAP, WSDL, y UDDI.

<https://www.infor.net>

¿Los Web Services pueden ser solo utilizados con HTTP?

A pesar de muchos limitar el uso de los Web Services al protocolo HTTP, los Web Services no fueron pensados para un protocolo en particular, es decir, nada nos impide utilizar SOAP sobre algún otro protocolo de Internet (SMTP, FTP, etc.). Se utiliza principalmente HTTP por ser un protocolo ampliamente difundido y que se encuentra menos restringido por firewalls (generalmente se bloquean puertos como el FTP, pero el HTTP es muy probable que no esté bloqueado).

¿Este no era un documento de INETWS?

Si este es un documento de como consumir el Web Service INETWS desde PHP, pero antes debía dar una introducción a lo que son los Web Services y cuáles son las partes que los componen. Creo que siempre es interesante y necesario poder saber que hay detrás de todo, para poder aprovechar al máximo las herramientas de productos o servicios, etc.

¿Qué es INETWS?

INETWS es una aplicación de Sistema a Sistema, más bien es un Web Service creado bajo el lenguaje PHP para Infor.net. Está compuesto por una serie de métodos que son moldeados por las funcionalidades que actualmente están en el sitio Web y que nos hará mucho más fácil el enlace entre sistemas. INETWS provee soporte para el desarrollo de clientes (aquellos que consumen los Web Services) y está basado en SOAP 1.1, WSDL 1.1 y HTTP 1.0/1.1

¿Cómo lo usamos?

Para responder a esta pregunta no hay nada mejor que un ejemplo de utilización de INETWS, el cual nos ayudara también a terminar de ver los conceptos mencionados anteriormente (SOAP, WSDL, etc.).

Este Web Service tiene 6 métodos que podemos utilizar (todos ellos se encuentran definidos en el archivo WSDL que se encuentra en https://right.infor.net/inetws/inetws.php?get_wsdl=1). Los métodos son: about(), busqueda_empresa(), busqueda_persona(), estudio_empresa(), estudio_persona() y UsernameToken().

Mensajes del Web Service

Como mencione anteriormente, toda la información del servicio Web INETWS se encuentra definida dentro del documento WSDL, para este Web Service se encuentra definida en:

https://right.infor.net/inetws/inetws.php?get_wsdl=1

Este documento en particular tiene 151 líneas, por lo que la lectura del mismo es bastante tediosa. No voy a explicar todo el archivo WSDL, sino que simplemente me detendré en las líneas que son de importancia para nuestra aplicación ejemplo (las que hacen referencia al método `busqueda_persona()`). Ahora veamos donde se encuentran las definiciones del método dentro del archivo:

Mensajes de `busqueda_persona`:

entre las líneas 17 a 26 podemos apreciar que se encuentra lo siguiente:

```
<message name="busqueda_persona">
  <part name="apellidos" type="xsd:string"/>
  <part name="nombres" type="xsd:string"/>
  <part name="orden" type="xsd:string"/>
  <part name="registro" type="xsd:string"/>
  <part name="pais" type="xsd:string"/> </message>
<message name="busqueda_personaResponse">
  <part name="busqueda_personaReturn" type="xsd:anyType"/> </message>
```

busqueda_persona: comencemos con las líneas 17 a 23

17. `<message name="busqueda_persona">`

En esta línea se indica que existe un método llamado `busqueda_persona`, que es el método que estábamos buscando.

18. `<part name="apellidos" type="xsd:string"/>`

Aquí se define que existe un parámetro para dicho método que se llama "apellidos" y es del tipo string.

19. `<part name="nombres" type="xsd:string"/>`

Aquí se define que existe un parámetro para dicho método que se llama "nombres" y es del tipo string.

20. `<part name="orden" type="xsd:string"/>`

Aquí se define que existe un parámetro para dicho método que se llama "orden" y es del tipo string.

21. `<part name="registro" type="xsd:string"/>`

Aquí se define que existe un parámetro para dicho método que se llama "registro" y es del tipo string.

22. `<part name="pais" type="xsd:string"/>`

Aquí se define que existe un parámetro para dicho método que se llama "pais" y es del tipo string.

23. `</message>`

Fin del método `busqueda_persona`

busqueda_personaResponse:

Ahora analicemos al método `busqueda_personaResponse`, como su nombre lo indica es la respuesta al método `busqueda_persona`. Es decir, cuando invocamos a `busqueda_persona`, el servidor nos responderá con un mensaje con el formato de `busqueda_personaResponse`. Las respuestas tienen anexo al final `Response` al nombre del método.

24. `<message name="busqueda_personaResponse">`

Esta línea nos indica que es una respuesta al método `busqueda_persona`

25. `<part name="busqueda_personaReturn" type="xsd:anyType"/>`

El método nos devuelve una variable "return" la cual es del tipo `anyType`. Como vimos anteriormente, los documentos WSDL pueden definir sus propios tipos de datos, `anyType` es un tipo de dato definido en este WSDL en el caso de este método es mixto ya sea de tipo string o `SOAPFault`

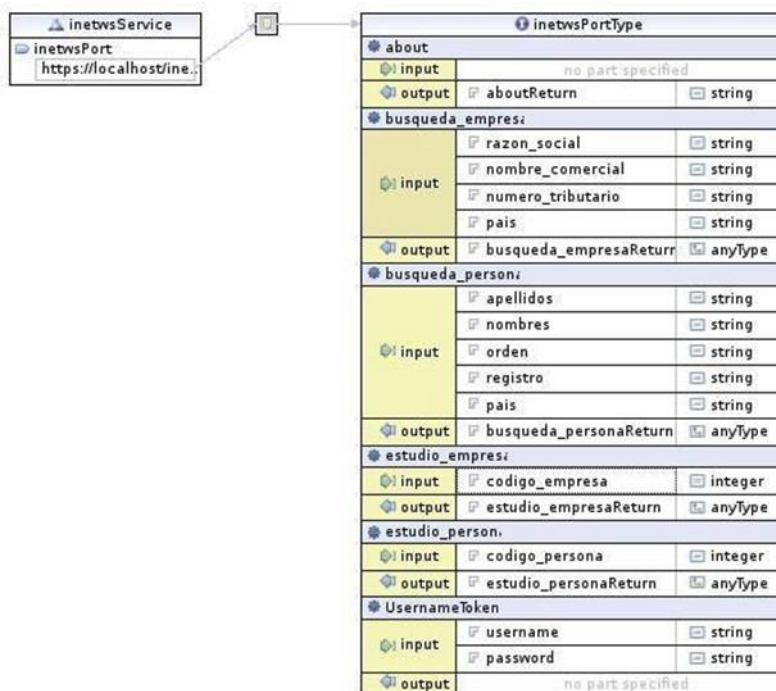
26. `</message>`

Fin del método `busqueda_persona`

Los demás métodos `busqueda_empresa()`, `estudio_persona()`, `estudio_empresa()`, `about()` y `UsernameToken()` tienen los mismos tipos de mensajes cada uno con sus parámetros y `Response`.

Comencemos a Programar

Como vemos no es fácil seguir todo lo que está definido dentro de un documento WSDL, por suerte muchas veces no nos tenemos que preocupar por todo esto ya que los mismos sitios donde se encuentran alojados los Web Services nos proveen de la información necesaria sin necesidad de tener que recurrir a ver los archivos WSDL. También existen aplicaciones que nos proveen de la información de manera gráfica. Ejemplo de INETWS:



Programación

Como mencione anteriormente la aplicación de prueba del Web Service solo es un ejemplo que desarrollaremos en el lenguaje PHP y para ello usaremos el método **busqueda_persona**.

Veamos como consumimos un Web Service (busqueda_persona):

En el código se ha simplificado para poder facilitar la lectura de los pasos. En su aplicación probablemente deseara incluir excepciones y/o manipulación de datos y manejo de errores.

Veamos cuáles son los pasos para consumir un servicio Web desde PHP:

```
require_once("soapUsernameToken.class");
```

Debemos incluir la librería donde está la clase SoapHeaderUsernameToken. En este ejemplo asumimos que el directorio donde se encuentra el archivo soapUsernameToken.class está localizado en el mismo directorio donde está nuestro script. La clase define un objeto de la cual se crea el encabezado de.

```
array(1) {  
[0]=> object(SoapHeader)#2 (4) {  
  ["namespace"]=> string(45) "http://schemas.xmlsoap.org/ws/2002/07/utility"  
  ["name"]=> string(13) "UsernameToken"  
  ["data"]=> object(SoapHeaderUsernameToken)#1 (2) {  
    ["Username"]=> string(7) "usuarioprueba"  
    ["Password"]=> string(10) "passwordprueba" }  
  ["mustUnderstand"]=> bool(false) } }
```

autenticación que veremos más adelante. Ver apéndice para ver el código de la clase.

```
ini_set("soap.wsdl_cache_enabled", "0");
```

Para propósitos de desarrollo se deshabilita el cache del documento WSDL. Esto es simplemente para no confundirnos con errores mientras se desarrolla la aplicación.

```
$u = 'usuarioejemplo';
```

```
$p = 'passwordejemplo';
```

En este ejemplo definimos explícitamente las variables \$u para el usuario y \$p para su password. En su aplicación debe crear un mecanismo de solicitud de usuario y password para que el usuario final lo ingrese manualmente.

```
$wsu = 'http://schemas.xmlsoap.org/ws/2002/07/utility';
```

En este paso usaremos el schema para establecer el encabezado de login

```
$usernameToken = new SoapHeaderUsernameToken($u, $p);
```

Instanciamos la clase SoapHeaderUsernameToken con los parámetros de \$u que representa usuarioejemplo y \$p que representa passwordejemplo


```
$soapHeaders[] = new SoapHeader($wsu, 'UsernameToken', $usernameToken);
```

Esta línea crea un arreglo en el formato de un encabezado de SOAP del objeto creado por la clase SoapHeaderUsernameToken bajo el schema de <http://schemas.xmlsoap.org/ws/2002/07/utility>. Suena a un montón, pero es simplemente la integración de las partes anteriores.

```
$wsdl = 'https://right.infor.net/inetws/inetws.php?get_wsdl=1';
```

Definimos aquí una variable para indicar dónde está el documento WSDL. El servicio Web INETWS tiene la siguiente característica. Buscando la mejor solución a nuestros clientes y para brindar un servicio constante, hemos creado un enlace inteligente en el sitio www.infor.net. Utilizando dicho enlace, al realizar la petición a http://www.infor.net/inetws/inetws.php?get_wsdl=1 pueden aprovechar nuestros servidores redundantes y canales encriptados. El enlace inteligente elige nuestro servidor más rápido del momento y establece una conexión segura con el usuario utilizando el protocolo HTTPS que cuenta con un certificado de seguridad respaldado por una entidad autorizada a nivel mundial. El documento WSDL que devuelve esta petición refleja el punto final del servidor adecuado.

```
$client = new SoapClient($wsdl);
```

Bueno finalmente aquí está la parte que nos interesa. La creación del cliente. Como puede observar para crear un objeto cliente SOAP ingresamos como parámetro el documento WSDL. Así de sencillo es debido a que todo lo demás está definido en los mensajes de dicho documento.

```
$client->__setSoapHeaders( $soapHeaders );
```

Si recuerda en líneas anteriores se creó la estructura del encabezado. Ahora es necesario establecer dicho encabezado para los mensajes del cliente SOAP.

```
$result = $client->busqueda_persona("perez perez", "juan jose", "DPI",  
"1101799045010", "GT");
```

Ya que esta creado el cliente y establecido el encabezado, llamemos a un método.

Para este ejemplo

utilizaremos `busqueda_persona()`. Los parámetros ingresados son los típicos de una búsqueda:

apellidos, nombres, cedula de Guatemala y país.

`header("Content-Type: application/xml; charset=UTF-8");`

Debido a que el resultado de la búsqueda, al igual que todos los métodos de INTEWS, es un archivo en formato XML, establecemos el encabezado para el protocolo HTTP como tal y definimos el carácter set como UTF-8 por los caracteres del idioma español.

`print $result;`

¿Que nos queda? Desplegar el resultado

SALIDA:

```
<?xml version="1.1" encoding="UTF-8"?>
<PERSONAS>
  <PERSONA>
    <CODIGO_PERSONA>2150350</CODIGO_PERSONA>
    <NOMBRE>PEREZ PEREZ, JUAN JOSE</NOMBRE>
    <SEXO>M</SEXO>
    <FECHA_NACIMIENTO>08/05/1970</FECHA_
NACIMIENTO> <EDAD>39</EDAD>
    <ORDEN>A-01</ORDEN>
    <REGISTRO>799045</REGISTRO>
    <ACODIGO_MUNICIPIO>GUA</ACODIGO_MUNICIPIO>
  <ACODIGO_PAIS>GTM</ACODIGO_PAIS>
  </PERSONA>
</PERSONAS>
```

Métodos de INETWS

Ahora que ya conocemos como sería el procedimiento para consumir el Web Service, veamos un poco más en profundidad los métodos del servicio Web INETWS.

Características de los métodos:

El método about() simplemente devuelve datos de la aplicación como nombre, copyright, autor, versión y licencia de uso que contiene los términos en que se presta el servicio y responsabilidades del usuario.

El método busqueda_empresa() busca a empresas es decir personas jurídicas. Este puede devolver múltiples resultados si fuera el caso dependiendo del criterio de búsqueda. Los datos que devuelve son:

<TIPO>

Es el tipo de código del propietario. Puede tener uno de dos valores 'S' o 'P' dependiendo si el código del propietario es una persona jurídica (Sociedad) o si es una persona individual.

<CODIGO>

Código numérico de la entidad encontrada

<PROPIETARIO>

Nombre del propietario

<NOMBRE_COMERCIAL>

Nombre comercial de la empresa

<NIT>

Numero tributario de la entidad

<DIRECCION>

Dirección de la entidad

<PAIS>

País origen de la entidad

El método tiene los siguientes parámetros de entrada.

- * @param string \$razon_social Razón Social de la empresa
- * @param string \$nombre_comercial Nombre Comercial de la empresa
- * @param string \$numero_tributario Numero tributario de la empresa
- * @param string \$pais País origen de la empresa

El método `búsqueda_persona()` busca a personas individuales. Este puede devolver múltiples resultados si fuera el caso dependiendo del criterio de búsqueda. Los datos que devuelve son:

<CODIGO_PERSONA>

Código numérico de la persona

<NOMBRE>

Nombre completo (apellidos, nombres) de la persona

<SEXO>

Género de la persona ('M','F')

<FECHA_NACIMIENTO>

Fecha de nacimiento (DD/MM/AAAA) de la persona

<EDAD>

Edad a la fecha de la persona

<ORDEN>

El orden de documento de identificación.

<REGISTRO>

Número de registro de documento de identificación

<ACODIGO_MUNICIPIO>

Municipio de origen de la persona

<ACODIGO_PAIS>

Abreviación de país origen de la persona

El método tiene los siguientes parámetros de entrada.

* @param string \$apellidos	Apellidos de la persona
* @param string \$nombres	Nombres de la persona
* @param string \$orden	Orden de documento de identificación
* @param string \$registro	Registro de documento de identificación
* @param string \$pais	País origen de la persona

El método estudio_empresa () devuelve los datos del estudio de una persona jurídica. Su único parámetro de entrada es el siguiente:

* @param int **\$codigo_empresa** Código de la empresa

Las secciones de información que devuelve son los siguientes (El detalle de cada uno de las secciones esta fuera del ámbito de este documento).

Etiquetas de información de la entidad:

<ESTUDIO_EMPRESA>

Etiqueta raíz del estudio

<FICHA_PRINCIPAL>

Etiqueta de datos generales de la entidad

<DIRECCIONES>

Direcciones de la entidad

<REFERENCIAS_JUDICIALES>

Etiqueta de referencias judiciales. Las referencias judiciales contienen dos etiquetas hijos. Estos son: **<DELITOS>** e **<INVOLUCRADOS>**.

<REFERENCIAS_PRENSA>

Etiqueta de referencias de prensa

<REFERENCIAS_COMERCIALES>

Etiqueta de referencias comerciales

<REFERENCIAS_MERCANTILES>

Etiqueta de referencias mercantiles.

<EMPRESAS_DE_SU_PROPIEDAD>

Etiqueta empresas de su propiedad

<VEHICULOS>

Etiqueta de vehículos

<INMUEBLES>

Etiqueta de inmuebles

<CONSULTAS_EFECTUADAS>

Etiqueta de consultas efectuadas

El método estudio_persona() devuelve los datos propios de una persona individual. El único parámetro de entrada es el siguiente:

* @param int **\$codigo_persona** Código de la persona

Las secciones de información que devuelve son los siguientes (El detalle de cada uno de las secciones esta fuera del ámbito de este documento).

Etiquetas de información de la persona:

<ESTUDIO_PERSONSA>

Etiqueta raíz del estudio

<FICHA_PRINCIPAL>

Etiqueta de datos generales de la persona

<DOCUMENTOS>

Documentos de identificación de la persona

<DIRECCIONES>

Direcciones de la persona

<PEP>

Persona Políticamente Expuesta

<PARIENTES_PEP>

Parientes de la Persona Políticamente Expuesta

<PARIENTES>

Parientes de la persona

<REFERENCIAS_JUDICIALES>

Etiqueta de referencias judiciales. Las referencias judiciales contienen dos etiquetas hijos. Estos son: **<DELITOS>** e **<INVOLUCRADOS>**.

<REFERENCIAS_PRENSA>

Etiqueta de referencias de prensa

<REFERENCIAS_COMERCIALES>

Etiqueta de referencias comerciales

<CHEQUES_GARANTIZADOS>

Etiqueta de cheques garantizados por el servicio de Infocheque.

<REFERENCIAS_MERCANTILES>

Etiqueta de referencias mercantiles.

<EMPRESAS_DE_SU_PROPIEDAD>

Etiqueta empresas de su propiedad

<EMPLEOS>

Etiqueta de empleos de la persona

<VEHICULOS>

Etiqueta de vehículos

<INMUEBLES>

Etiqueta de inmuebles

<CONSULTAS_EFECTUADAS>

Etiqueta de consultas efectuadas

El método UsernameToken() es llamada desde el encabezado SOAP del mismo nombre. El nombre de la función es una etiqueta auth, estándar de seguridad en servicios Web. Este sirve para poder identificar y autenticar al usuario utilizando los métodos del servicio. El método tiene los siguientes parámetros de entrada.

* @param string \$username

Usuario del cliente

* @param string \$password

Password del usuario

Cosas para tomar en consideración.

Para los usuarios que conocen y han utilizado el sitio Web quizás ya aprovechan estos consejos. Consejos de Búsqueda

Para consulta de personas, podrá hacerlas por apellidos y nombres y por documento. Al buscar por documento es necesario ingresar al menos dos parámetros en el método de `busqueda_persona()`. Estos son “orden” y “registro”. Orden es el tipo de documento y Registro es el número del documento. El parámetro Orden puede ser uno de los siguientes valores:

Valores de orden

DPI = DOCUMENTO PERSONAL DE IDENTIFICACION

NIT = ID. TRIBUTARIA GUATEMALA

LCA = LICENCIA DE CONDUCIR GUATEMALA

PAS = PASAPORTE GUATEMALA

Cuando busca a una persona por nombres y apellidos es recomendable ingresar el parámetro

“pais”. Esto acelera la búsqueda. El país se refiere al lugar de nacimiento de cada persona y no a la nacionalidad. Una persona puede tener más de una nacionalidad. Si no se conoce el país origen de una persona se debe dejar vacío este parámetro para buscar en todos los países. El parámetro País puede ser uno de los siguientes valores:

Valores de país

CA CANADA

CO COLOMBIA

CR COSTA RICA

EC ECUADOR

SV EL SALVADOR

US ESTADOS UNIDOS DE AMERICA GT GUATEMALA

HN HONDURAS

MX MEXICO

NI NICARAGUA

PA PANAMA

TODOS LOS PAISES

Ejemplos:

Búsqueda por nombres y apellidos

```
busqueda_persona("perez perez","juan jose","","."GT");
```

Búsqueda por DPI de Guatemala

```
busqueda_persona("","DPI","CUI1999995620101"."");
```

Para consultar empresas podrá hacerlo por razón social, nombre comercial y numero tributario.

Al buscar por número tributario es necesario ingresar el parámetro "numero_tributario" en al método de busqueda_empresa(). Para buscar por razón social o nombre comercial es recomendable ingresar el parámetro "pais" para acelerar la búsqueda. El país se refiere al lugar donde está constituida la empresa. Si no se conoce el país de una empresa se debe dejar vacío este parámetro para buscar en todos los países. Los posibles valores del parámetro País del método busqueda_empresa() son iguales al método de busqueda_persona().

Uso de Comodines

El uso de los comodines nos sirve para búsquedas más amplias y pueden ayudarle a encontrar nombres incompletos y/o mal escritos.

Los comodines se pueden usar en los parámetros de nombres y apellidos en el método busqueda_persona() o razón social y nombre comercial en el método busqueda_empresa(). Hay tres comodines que puede usar. Estos son el carácter '\' que puede reemplazar una Ñ o ñ, el carácter '_' para reemplazar un carácter o espacio y el carácter '%' que puede representar uno o varios caracteres. La utilización de comodines hace más tardada la búsqueda y no son necesarios si tiene los datos para el criterio de búsqueda correctos. Un ejemplo de búsqueda con el comodín '_' es si no está seguro de como se escribe un apellido, con Z o S, podrá utilizar en lugar de cada letra que le dé duda el carácter '_'. Para buscar una persona de apellido VALDEZ o VALDES puede ingresar 'VALDE_'. es si desconoce cómo se escribe un nombre o apellido que se escribe con la letra LL o Y, o R o RR deberá utilizar el comodín, para búsquedas más amplias '%'. Para buscar AYERDI o ALLERDI puede ingresar 'A%ERDI'.

¿Cómo lo podemos usar?

Los métodos mencionados se pueden usar conjuntamente. La búsqueda de una persona o una empresa devuelve entre otras cosas el código de la misma, con este código se puede llamar al estudio correspondiente y obtener toda la información disponible de la entidad. Existen muchas formas en que podríamos usar este Web Service, por ejemplo el usuario puede buscar a una persona, seleccionarlo, ver su estudio, tomar los códigos de las entidades que están involucrados en la una referencia judicial y consecuentemente ver los estudios de cada uno de ellos o ver los estudios de las empresas de las cuales es representante. En fin, se puede utilizar de tantas maneras que enriquecerá el uso de la información.

Conclusión

EL Web Service INETWS es un tema apasionante, ya que brindan gran funcionalidad y posibilidad de realizar cambios y mejorar nuestras aplicaciones, complementando las operaciones, creando oportunidades y reduciendo el riesgo de su negocio. Un mismo Web Service puede ser consumido tanto por aplicaciones que se ejecutan en un servidor y entregan código HTML al cliente, como por aplicaciones que corren directamente en la máquina del cliente. Como vemos las posibilidades son infinitas, es solo cuestión de comenzar a desarrollar.

Apéndice:

Atención al Cliente

INFORMES EN RED GUATEMALA, S.A.

Ciudad Guatemala

24 calle 347 Zona 1,

Tel (502) 2412 7444

Fax (502) 22300956

Pbx. (502)22214202

Administración

Asuntos administrativos:
administracion@infor.net

Mercadeo

Contratación del servicio:
mercadeo@infor.net

Referencias

Contribución de referencias crediticias positivas o negativas:
referencias@infor.net

Actualizaciones

Modificaciones, reclamos, nueva información.:
actualizaciones@infor.net

Soporte de Sistemas IT

Asuntos relacionados a resolver problemas con servicios integrados de Sistemas:
soporte@infor.net

Apéndice: Código soapUserNameToken.class

```
<?php
/**
 * SOAP Header Username Token class for inetws
 *
 * @author Gabriel Paz <gabriel.paz@infor.net>
 * @copyright 2016 Gabriel Paz
 * @version 3.0
 *
 * @throws SoapFault *
 */
class SoapHeaderUsernameToken{
    /** @var int
     Username */
    public
    $Username;
    /** @var int Password */
    public $Password;

    public function
    __construct($l, $p){
        $this->Password =
        $p;
        $this-
        >Username = $l;
    }
}
?>
```

Apéndice: Código inetws.client.php

```
<?php
/**
 * inetws SOAP client
 * @author Gabriel Paz <gabriel.paz@infor.net>
 * @copyright 2016 Gabriel Paz
 * @version 3.0 *
 * @throws SoapFault
 */

require_once('soapUserName
Token.class');
try {
ini_set("soap.wsdl_cache_enabled", "0");

$u = 'usuarioejemplo';
$p = 'passwordejemplo';

// Set the login headers
$wsu = 'http://schemas.xmlsoap.org/ws/2002/07/utility';
$usernameToken = new SoapHeaderUsernameToken($u, $p);
$soapHeaders[] = new SoapHeader($wsu, 'UsernameToken', $usernameToken);

$wsdl = 'http://www.infor.net/inetws/inetws.php?get_wsdl=1';

$client = new SoapClient($wsdl);
$client->__setSoapHeaders( $soapHeaders );

//Busqueda persona
//APELLIDOS, NOMBRES, ORDEN, REGISTRO, CODIGO_PAIS
$result = $client->busqueda_persona("perez perez", "juan jose", "DPI", "1633540320101", "GT");

//Busqueda empresa
//RAZON_SOCIAL, NOMBRE_COMERCIAL, NUMERO_TRIBUTARIO,
CODIGO_PAIS //result = $client->busqueda_empresa("", "", "", "", "");

//Estudio persona
//CODIGO_PERSONA
//result = $client->estudio_persona();
//result = $client->estudio_persona(19781393);

//Estudio empresa
//CODIGO_EMPRESA
//result = $client->estudio_empresa();
//result = $client->estudio_empresa(3610637);
```

```
//About
//$result = $client->about();

header("Content-Type:          application/xml;
charset=UTF-8"); print $result;

} catch(SoapFault $f) {
    print "Algo salio mal al ejecutar su peticion: {$f-
>getMessage()}";
}
?>
```

CODIGOS DE VERIFICACION ERROR

00001</numero><mensaje>Debe de ampliar su selección
 00002</numero><mensaje>Ninguna entidad encontrada.
 00003</numero><mensaje>Verificar su acceso.
 00004</numero><mensaje>El usuario a llegado al límite de consultas.
 00005</numero><mensaje>Esta información solo puede ser consultada previa autorización
 del titular por un acuerdo voluntario realizado en la procuraduría de derechos humanos o
 por orden judicial
 00006</numero><mensaje>El titular ha manifestado que no se divulgue información
 acerca de el sin su autorización

Apéndice: Revisión Histórica

Nombre	Fecha	Descripción de la revisión	Versión
Gabriel Paz		31/03/2018 Revisión general del Webservices	1.4
Gabriel Paz		31/03/2020 Revisión general del Webservices	1.5
Gabriel Paz		21/01/2021 Revisión general del Webservices	1.6
Gabriel Paz		24/02/2021 Pep(Persona Políticamente expuesta)	
Gabriel Paz		04/08/2022 Revisión general del WebServices	2.0
Gabriel Paz		31/03/2024 Revisión General del WebServices	3.0